

Comunicação Convergente: do Chat à Voz

Fundamentação Teórica e Tutorial de Instalação
WebSocket + SIP/WebRTC + Asterisk em containers Docker

Lucas Alves

Orientadora: Luciana Pereira Oliveira

Disciplina: Redes Convergentes — IFPB

2026

Resumo

Este documento apresenta a fundamentação teórica e o tutorial prático de um projeto de *redes convergentes*: uma aplicação web em que dois usuários conversam por texto em tempo real (**WebSocket**) e, com um único clique, *escalam* a conversa para uma chamada de voz (**SIP/WebRTC**) orquestrada pelo **Asterisk**. Todo o ambiente é empacotado em **containers Docker**, garantindo portabilidade e reprodutibilidade. A Parte I cobre os conceitos (funcionamento, arquitetura, aplicações, vantagens e limitações) e a Parte II traz o passo a passo de instalação, configuração e uso.

Sumário

1	Introdução e tema	1
	Parte I — Fundamentação teórica	1
2	Do HTTP ao tempo real: WebSocket	1
3	Voz sobre IP: SIP, RTP e WebRTC	2
3.1	SIP e RTP	2
3.2	WebRTC	2
3.3	SIP sobre WebSocket: a ponte	2
4	Asterisk e a interface de gerência (AMI)	2
5	Containers: portabilidade e reprodutibilidade	3
6	Arquitetura do projeto	3
7	Aplicações no mercado de trabalho	3
8	Vantagens e limitações da solução	3
	Parte II — Tutorial de instalação e configuração	4
9	Pré-requisitos	4
10	Obtenção do projeto	4
11	Estrutura de diretórios	4

12 Componentes comentados	4
12.1 Orquestração (docker-compose.yml)	4
12.2 Asterisk	4
12.3 Servidor de chat (server.js)	5
12.4 Front-end (JsSIP)	5
13 Subir o ambiente	5
14 Verificação	5
15 Uso	5
16 Experimento e coleta de métricas	6
17 Solução de problemas (<i>troubleshooting</i>)	6
18 Conclusão	6

1 Introdução e tema

Convergência de redes é a tendência de transportar voz, dados e vídeo sobre uma mesma infraestrutura IP, em vez de redes separadas para cada serviço. O projeto aqui documentado é um exemplo didático dessa convergência: uma única aplicação web faz conviver **dois meios de comunicação distintos** — mensageria em tempo real (texto) e voz — e permite a transição transparente de um para o outro.

O cenário é familiar no mercado: muitos atendimentos começam por chat e, quando o texto não basta, “sobem” para uma ligação. Reproduzir isso exige combinar tecnologias de mundos diferentes: o WebSocket (web/dados) e o SIP/WebRTC (telefonia/voz). É exatamente essa *integração* que caracteriza um sistema convergente.

Parte I — Fundamentação teórica

2 Do HTTP ao tempo real: WebSocket

O **HTTP** é um protocolo *requisição-resposta* e sem estado: o cliente pergunta, o servidor responde, e a conexão se encerra. Esse modelo é ótimo para páginas e APIs, mas ruim para comunicação *bidirecional em tempo real*, pois o servidor não consegue enviar dados ao cliente por iniciativa própria. As soluções antigas (*polling* e *long-polling*) ficam perguntando “tem novidade?” repetidamente, desperdiçando rede e adicionando latência.

O **WebSocket** (RFC 6455) resolve isso. A conexão começa como um HTTP comum e é *promovida* (*upgrade*) para um canal persistente e full-duplex sobre a mesma porta TCP. O handshake é uma requisição com os cabeçalhos **Connection: Upgrade** e **Upgrade: websocket**, à qual o servidor responde com **101 Switching Protocols**. A partir daí, cliente e servidor trocam *frames* livremente, nos dois sentidos, com pouquíssima sobrecarga por mensagem.

Aplicações. Chats, dashboards ao vivo, jogos multiplayer, edição colaborativa, notificações *push* e cotações financeiras.

Vantagens. Comunicação bidirecional de baixa latência; menor sobrecarga que o *polling*; usa as portas 80/443, atravessando a maioria dos *firewalls*.

Limitações. A conexão tem estado, o que complica o balanceamento de carga e a escalabilidade horizontal; alguns *proxies* antigos não suportam o *upgrade*; e perde-se a semântica de cache e métodos do HTTP.

3 Voz sobre IP: SIP, RTP e WebRTC

3.1 SIP e RTP

A **VoIP** (Voz sobre IP) transporta áudio pela rede de dados. Para isso são necessários dois protocolos com papéis distintos:

- **SIP** (*Session Initiation Protocol*, RFC 3261): protocolo de *sinalização*. É ele que registra os ramais (REGISTER), inicia chamadas (INVITE), e as encerra (BYE). O SIP *não* carrega o áudio: ele negocia *onde* e *como* o áudio vai trafegar (via SDP).
- **RTP** (*Real-time Transport Protocol*): protocolo que de fato carrega os pacotes de áudio/vídeo, geralmente sobre UDP, com numeração de sequência e marca de tempo para reconstruir o fluxo.

3.2 WebRTC

O **WebRTC** leva a comunicação de mídia em tempo real para *dentro do navegador*, sem plugins. Seus pilares:

- **getUserMedia**: captura o microfone/câmera (exige *contexto seguro*: HTTPS ou localhost).
- **DTLS-SRTP**: a mídia é *obrigatoriamente* cifrada — não existe WebRTC “em claro”.
- **ICE/STUN/TURN**: descobrem caminhos de rede para atravessar NAT/*firewall*.
- Codecs de áudio: **Opus** (padrão, banda variável) e **G.711** (*ulaw/alaw*, 64 kbps).

3.3 SIP sobre WebSocket: a ponte

Como unir o navegador (WebRTC) ao mundo SIP? Transportando a *sinalização SIP dentro de um WebSocket*. Uma biblioteca JavaScript como o **JsSIP** transforma o navegador em um *softphone*: ela abre um WebSocket para o servidor SIP e troca mensagens SIP por ele, enquanto o áudio segue por WebRTC (DTLS-SRTP). É essa combinação que permite uma chamada de voz *100% no navegador*, sem instalar nada.

4 Asterisk e a interface de gerência (AMI)

O **Asterisk** é o *PBX* (central telefônica) de código aberto mais usado do mundo. No projeto, ele atua como o servidor SIP: registra os ramais, faz a ponte entre as chamadas e fala WebRTC com os navegadores através do módulo `chan_pjsip` com transporte WebSocket.

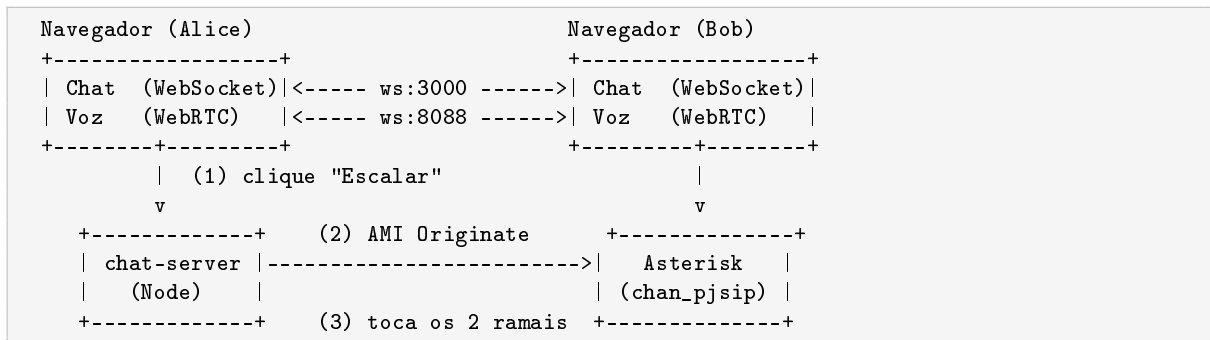
Além do SIP, o Asterisk expõe a **AMI** (*Asterisk Manager Interface*), uma interface de gerência por TCP. Por ela, uma aplicação externa pode comandar o Asterisk — por exemplo, emitir um **Originate** para iniciar uma chamada entre dois ramais. É esse comando que implementa a “escalada” do chat para a voz: o servidor de chat, ao receber o clique, pede ao Asterisk que ligue os dois usuários (*click-to-call*).

5 Containers: portabilidade e reprodutibilidade

O **Docker** empacota cada serviço (Asterisk, servidor de chat) com todas as suas dependências em uma *imagem*, que roda igual em qualquer máquina. O **docker-compose** descreve e sobe todo o conjunto com um comando. O ganho é direto para um trabalho acadêmico: o avaliador clona o repositório e reproduz o ambiente *idêntico*, sem “na minha máquina funciona”. Essa é a base do critério de *reprodutibilidade*.

6 Arquitetura do projeto

A figura a seguir resume os componentes e o fluxo da escalada:



Há **dois canais** convivendo: o chat trafega pelo WebSocket do **chat-server** (porta 3000); a voz trafega pelo WebSocket SIP do Asterisk (porta 8088) e pela mídia WebRTC. O clique de escalada vai pelo canal de dados; a chamada resultante vem pelo canal de voz — a convergência em ação.

7 Aplicações no mercado de trabalho

- **Contact centers e plataformas omnichannel:** o atendimento começa por chat e escala para voz no mesmo fluxo (Zendesk, Twilio Flex, etc.).
- **Click-to-call** em sites de e-commerce e suporte (“fale com um atendente”).
- **Colaboração e conferência:** Microsoft Teams, Google Meet, Discord e Jitsi usam WebRTC para voz/vídeo no navegador.
- **Telemedicina e helpdesk:** chat clínico/técnico que vira teleconsulta.
- **Telefonia corporativa:** o Asterisk é amplamente usado como PBX/IP-PBX em empresas e provedores.

8 Vantagens e limitações da solução

Vantagens.

- Uma única *stack* web atende dados e voz.
- Voz no navegador, sem plugin ou *softphone* externo.
- Mídia sempre cifrada (DTLS-SRTP).
- Ambiente portátil e reproduzível via containers.

Limitações.

- O WebRTC exige *contexto seguro* (HTTPS/localhost) e, em produção, servidores TURN para atravessar NAT.
- A configuração de WebRTC no Asterisk tem curva de aprendizado (transporte, DTLS, ICE).
- O uso de `network_mode: host` (que simplifica a mídia no laboratório) é restrito ao Linux.

Parte II — Tutorial de instalação e configuração

9 Pré-requisitos

- Sistema operacional **Linux** (por causa do `network_mode: host`).
- **Docker** e **docker-compose** instalados.
- Acesso de *root*/sudo ao Docker.
- Um **navegador** moderno (Chrome/Firefox) com microfone.
- Portas livres: 3000 (front/chat), 8088 (SIP/WS), 5038 (AMI) e 10000-10100/udp (RTP).

10 Obtenção do projeto

```
git clone git@github.com:lcsg7/redes-convergentes.git
cd redes-convergentes
```

11 Estrutura de diretórios

```
redes-convergentes/
|- docker-compose.yml      # orquestra os 2 servicos
|- asterisk/
|   |- Dockerfile          # Ubuntu 22.04 + Asterisk 18
|   \- config/             # http, pjsip (WebRTC), dialplan, AMI, modules
|- chat-server/
|   |- Dockerfile          # Node 20
|   | server.js            # chat (WebSocket) + AMI Originate + coletor CSV
|   \- public/             # front: index.html, app.js (JsSIP), vendor/
|- scripts/netem.sh        # perfis de degradacao de rede (tc/netem)
|- analysis/analise.ipynb  # notebook dos graficos
\ metrics/                 # saida das medicoes (metrics.csv)
```

12 Componentes comentados

12.1 Orquestração (docker-compose.yml)

Sobe dois serviços em `network_mode: host` (no Linux, navegador e Asterisk se enxergam em 127.0.0.1, o que simplifica a mídia WebRTC). O `chat-server` monta o front e o `server.js` como volumes, além do diretório `metrics` (compartilhado com o host).

12.2 Asterisk

Pontos-chave da configuração em `asterisk/config/`:

- `http.conf`: habilita o servidor HTTP em :8088, onde fica o endpoint WebSocket SIP (`/ws`).
- `pjsip.conf`: define o transporte WebSocket e dois *endpoints* WebRTC (`alice` e `bob`) com `webrtc=yes` e `dtls_auto_generate_cert=yes`.
- `extensions.conf`: o plano de discagem que liga os ramais.
- `manager.conf`: o usuário AMI usado pelo `chat-server` para o `Originate`.
- `modules.conf`: **desabilita o `chan_sip`** — detalhe crítico, explicado na seção de solução de problemas.

12.3 Servidor de chat (server.js)

Um servidor Node que: (i) serve o front; (ii) mantém o chat por WebSocket; (iii) ao receber o evento de escalada, emite um **Originate** via AMI para o Asterisk; e (iv) registra as métricas de cada chamada (tempo de estabelecimento e duração) em `metrics/metrics.csv`.

12.4 Front-end (JsSIP)

A página carrega o **JsSIP** (hospedado localmente em `public/vendor/`), que registra o navegador como ramal WebRTC no Asterisk e atende as chamadas. O mesmo HTML mantém o chat por um WebSocket nativo — os dois canais lado a lado.

13 Subir o ambiente

```
docker-compose up --build      # use sudo se o Docker exigir
```

O primeiro *build* baixa o Ubuntu/Asterisk e as dependências do Node (precisa de internet). Ao final, os dois containers ficam *Up*.

14 Verificação

```
# raramente alice/bob existem?
docker exec conv-asterisk asterisk -rx "pjsip show endpoints"

# o endpoint WebSocket /ws responde ao handshake? (espera 101 + protocol sip)
curl -i -N -H "Connection: Upgrade" -H "Upgrade: websocket" \
  -H "Sec-WebSocket-Version: 13" -H "Sec-WebSocket-Key: dGhlc2FtcGx1" \
  -H "Sec-WebSocket-Protocol: sip" http://localhost:8088/ws

# o servidor de chat conectou no AMI?
docker logs conv-chat-server
```

15 Uso

1. Abra `http://localhost:3000` em **duas** janelas.
2. Entre como **Alice** em uma e **Bob** na outra; permita o microfone.
3. Confirme o indicador **“SIP: registrado”** no topo.
4. Troque mensagens de texto (canal WebSocket).
5. Clique em **“Escalar para ligação”**: os dois navegadores tocam e atendem automaticamente — agora é voz (canal SIP/WebRTC).

Dica: na mesma máquina, use fone de ouvido (os dois lados dividem o microfone/alto-falante e geram eco).

16 Experimento e coleta de métricas

Cada chamada encerrada grava uma linha em `metrics/metrics.csv` (`timestamp,tag,setup_ms,duration_s`). O perfil de rede ativo é controlado pelo script `netem.sh`, que degrada a interface `lo`:

```
sudo bash scripts/netem.sh apply 100 0 delay100 # atraso de 100 ms
sudo bash scripts/netem.sh apply 0 5 loss5      # perda de 5%
sudo bash scripts/netem.sh clear                # remove a degradacao
```

Para cada perfil, aplique o `netem` e rode dezenas de escaladas; depois abra `analysis/analise.ipynb` (Jupyter ou o JupyterLite do GPSC) para gerar os gráficos de tempo de estabelecimento, duração e banda.

17 Solução de problemas (*troubleshooting*)

- **SIP não registra / WebRTC não conecta:** provavelmente o `chan_sip` (driver SIP antigo) está roubando o sub-protocolo WebSocket `sip` antes do PJSIP. A correção é o `noload => chan_sip.so` no `modules.conf` (já incluído).
- **Microfone bloqueado:** acesse por `http://localhost` (contexto seguro); por IP o navegador bloqueia a captura.
- **Asterisk não instala no Debian:** o pacote foi removido do Debian; por isso a imagem usa `ubuntu:22.04`.
- **Erro KeyError: 'ContainerConfig' ao recriar:** é um *bug* do docker-compose v1 com o Docker novo. Contorne com `docker-compose down && docker-compose up -d --build`.

18 Conclusão

O projeto demonstra, de ponta a ponta, a convergência de *dados* e *voz* sobre IP: um chat em tempo real (WebSocket) que escala para uma chamada de voz (SIP/WebRTC) sem sair do navegador, com todo o ambiente reproduzível em containers. Os conceitos teóricos (WebSocket, SIP, WebRTC, Asterisk) se materializam num cenário prático e mensurável, alinhado aos objetivos da disciplina de Redes Convergentes.

Referências

- [1] FETTE, I.; MELNIKOV, A. *RFC 6455 — The WebSocket Protocol*. IETF, 2011.
- [2] ROSENBERG, J. et al. *RFC 3261 — SIP: Session Initiation Protocol*. IETF, 2002.
- [3] W3C. *WebRTC: Real-Time Communication in Browsers*. <https://www.w3.org/TR/webrtc/>.
- [4] SANGOMA. *Asterisk Documentation*. <https://docs.asterisk.org/>.
- [5] VERSATICA. *JsSIP — The JavaScript SIP library*. <https://jssip.net/>.
- [6] DOCKER INC. *Docker Documentation*. <https://docs.docker.com/>.